Building and Analyzing Monolingual Transformer Language Models in Hindi and Nepali

1 Project Overview

Hindi (Model H) represents the higher-resource setting, while Nepali (Model L) represents the lower-resource setting.

Table 1: Experimental setup for the two models.

	Hindi (H)	**Nepali (L)**
Resource tier	Higher	Lower
Training tokens	≈500M	≈500M
Vocabulary	10,000	10,000
Parameters	25.19M	25.19M
Layers	12	12
Attention heads	6	6
d_{model}	384	384
Context length	512	512

2 Corpus Construction and Tokenization

2.1 Data Collection and Cleaning

For Hindi, I used Sangraha Verified as the public source, Wikipedia and Wikisource as the manual source from where webscraping was done. For Nepali, I used Sangraha Verified as the public source and manually scraped from Wikimedia and news articles. The cleaning pipeline consists of Unicode normalization, MediaWiki Markup Cleanup (Manual Only), Character/Script Filtering, Length Filtering (Manual Only), Line Structure Filtering (Manual Only), Repetition Filtering (Manual Only), Latin Ratio Filtering. Then i did exact deduplication using SHA-256 hashing, and near-duplicate detection using MinHash and locality-sensitive hashing.

After cleaning, Hindi decreased from 634.02M to 610.35M words, and Nepali decreased from 541.15M to 534.04M words. The manual portions decreased relative more because the length, line-structure, and repetition filters were applied to those collections.

No exact duplicates were removed in both languages. For Hindi, near-deduplication removed 22,345 documents, and 8,593 documents were removed for Nepali.

2.2 Final Corpora

Table 2: Final deduplicated corpus statistics.

	Hindi	**Nepali**
Documents	1,482,739	1,730,416
Words	608.09M	529.93M
Characters	3.13B	3.41B
Manual document fraction	17.74%	9.15%
Public document fraction	82.26%	90.85%

I then created the final training sets separately for Hindi and Nepali, with 500M tokens for each language. The final Hindi training set contained 500,000,728 tokens, and the Nepali training set contained 500,000,022 tokens.

2.3 Tokenizer Selection

A separate BPE tokenizer was trained from scratch for each language. Three vocabulary sizes 5K, 7.5K, and 10K were evaluated on the validation data.

Table 3: Tokenizer comparison on the validation split.

Language	Vocabulary	Fertility	Chars/token	UNK rate
Hindi	5K	2.5457	2.0172	6.57×10^{-8}
Hindi	7.5K	2.4340	2.1098	6.87×10^{-8}
Hindi	**10K**	**2.3735**	**2.1635**	7.05×10^{-8}
Nepali	5K	2.6695	2.4128	7.09×10^{-8}
Nepali	7.5K	2.5268	2.5490	7.49×10^{-8}
Nepali	**10K**	**2.4483**	**2.6308**	7.73×10^{-8}

In both languages, as vocabulary size increases fertility decreases and average number of characters represented by each token increases. The unknown-token rates were extremely small for all three choices. Therefore, 10K was selected as the final vocabulary for both languages, as it gave the best tokenization efficiency among the three vocabulary sizes.

3 Model Architecture and Pretraining

Table 4: Transformer architecture.

Parameter	Value
Vocabulary size	10,000
Model dimension	384
Number of layers	12
Number of heads	6
Head dimension	64
FFN dimension	1,536
Activation	GELU
Context length	512
Positional encoding	RoPE
Normalization	Pre-LayerNorm
Dropout	0.1
Embedding/output weights	Tied
Trainable parameters	25,190,400

For pretraining, i used a micro-batch size of 16, gradient accumulation of 8, and a 512-token context. so, each optimizer update represented 65,536 tokens. The training was done for 7,630 optimizer steps, which is approximately one pass over 500M training tokens.

3.1 Training Behaviour

Both models showed stable convergence. Training and validation losses decreased during training without any divergence between them during the last steps.

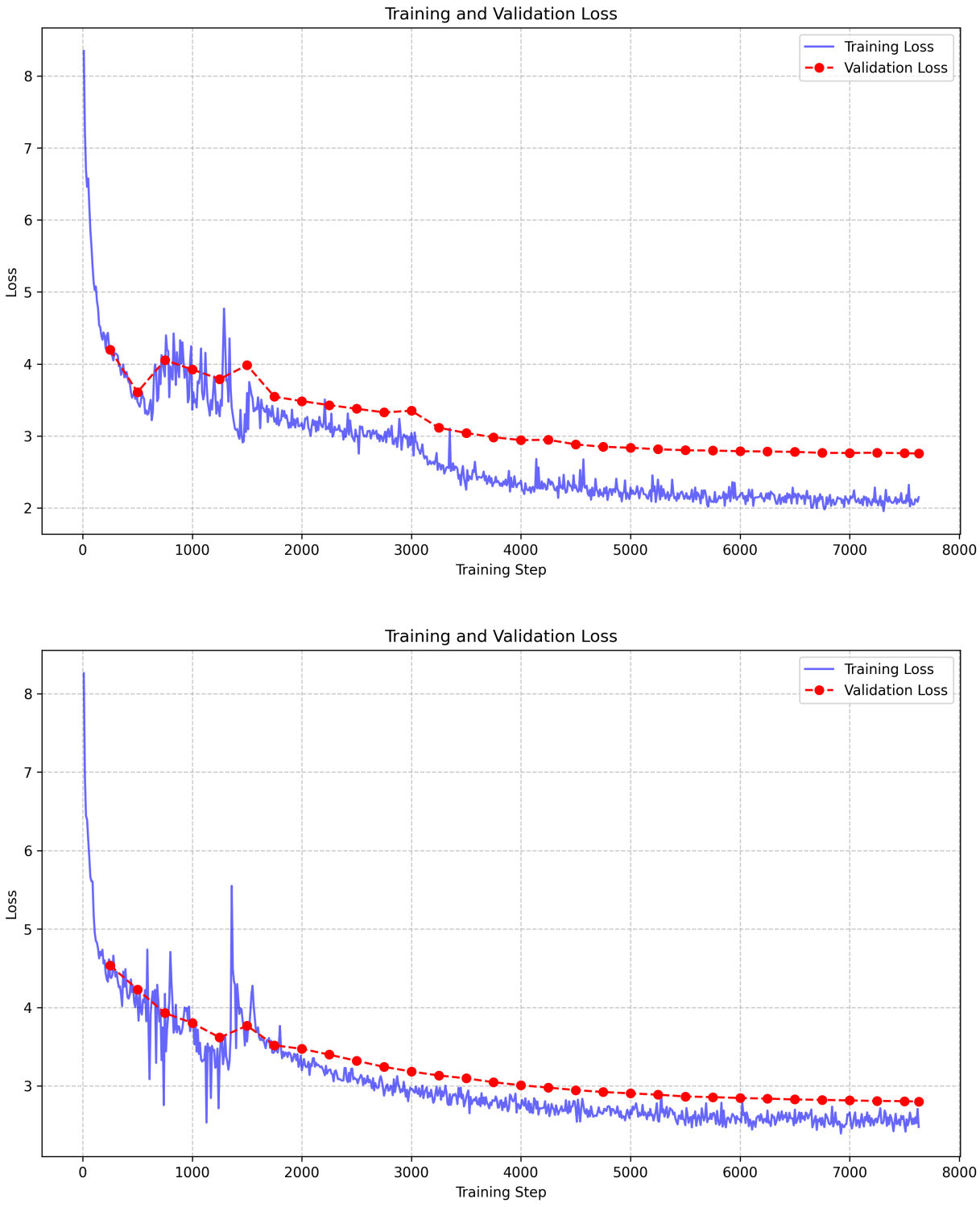


Figure 1: Training and validation loss curves for the Hindi and Nepali language models during pretraining.

4 Language-Modeling Results

4.1 Intrinsic Evaluation

Table 5: Intrinsic language-modeling results.

Language	Split	CE	PPL	BPB	Pred. tokens
Hindi	Validation	2.4411	11.4863	0.6383	14.18M
Hindi	Test	2.4565	11.6644	0.6441	14.40M
Nepali	Validation	2.5288	12.5384	0.5207	12.93M
Nepali	Test	2.5265	12.5099	0.5206	12.86M

On test data, Hindi has a lower token-level perplexity (11.66 versus 12.51). However, Nepali has a lower BPB (0.5206 versus 0.6441). we can say that perplexity is calculated over model tokens and hence depends strongly on how each tokenizer segments the text.

4.2 Tokenizer Efficiency During Evaluation

The final 10K tokenizers also showed difference in their measured efficiency on held-out data.

Table 6: Held-out tokenizer efficiency.

Language	Tokens/word	Bytes/token
Hindi	2.3782	5.5019
Nepali	2.4494	7.0014

Nepali has higher fertility than Hindi, so its words are fragmented into more subword tokens on average.

4.3 Generation Quality

These metrics were calculated by generating 500 examples from test.jsonl for each language. A prefix length of 64 tokens was used, and generation was stopped at a maximum of 64 new tokens.

Table 7: Hindi generation metrics.

Config	BLEU	chrF	ROUGE-L	RepRate	Dist-1	Dist-2
Greedy	2.0919	13.4348	0.1121	0.6114	0.0701	0.2031
T=0.5	2.0552	15.6424	0.1173	0.2071	0.1181	0.4261
T=1.0	0.9423	15.6740	0.0872	0.0110	0.2257	0.8038
T=1.5	0.0733	15.2115	0.0364	0.0000	0.2555	0.9660

Table 8: Nepali generation metrics.

Config	BLEU	chrF	ROUGE-L	RepRate	Dist-1	Dist-2
Greedy	1.8775	17.8625	0.0827	0.5008	0.1024	0.2906
T=0.5	1.9386	20.4337	0.0935	0.1295	0.1672	0.5682
T=1.0	0.7900	19.5160	0.0584	0.0045	0.2620	0.8716
T=1.5	0.2056	18.3791	0.0299	0.0000	0.2851	0.9704

Increasing the temperature reduced repetition and increased the lexical diversity. For example, Hindi repetition rate decreased from 0.6114 under greedy decoding to 0.0110 at temperature 1.0, but Distinct-2 increased from 0.2031 to 0.8038.

5 Reasoning Fine-Tuning

5.1 Synthetic Dataset

The reasoning dataset was generated seperately for Hindi and Nepali. Each language contains 7,000 examples where 5,600 are train, 700 val, and 700 test examples.

Seven different task families were used: direct comparison, largest/smallest, general comparison, two-hop transitivity, three-hop transitivity, reversed relation, and mixed equality.

The dataset was balanced across the seven task families, with 1,000 examples per family. I have chosen four template variations for each family in each language, so 28 Hindi templates and 28 Nepali templates. Entities were different between training, validation, and test splits so that test entities were not seen during training.

The dataset had different attributes such as height, price, age, and quantity, and also had easy, medium, and hard examples.

5.2 Fine-Tuning Setup

During fine-tuning I used LoRA while keeping the Phase 2 base weights frozen. The adapters were mainly placed at query, key, value, and output projections.

Table 9: LoRA fine-tuning configuration.

Parameter	Value
LoRA rank	8
LoRA alpha	16
LoRA dropout	0.05
Target modules	W_q, W_k, W_v, W_o
Trainable parameters	294,912 (1.16%)
Epochs	5
Micro-batch size	2
Gradient accumulation	8
Effective batch size	16
Learning rate	2×10^{-4}
Weight decay	0.01
Optimizer	AdamW
Scheduler	Cosine with warmup
Precision	FP16 AMP
Loss	Target/answer tokens only

Loss was computed only on the target answer tokens.

6 Reasoning Results

6.1 Overall Performance

The pretrained models performance on exact-match reasoning was almost zero. After SFT, both model's task-specific reasoning performance increased.

Table 10: Pretrained versus SFT reasoning performance on 700 test examples.

Language	Model	Reasoning Accuracy	Exact Match
Hindi	Pretrained	0.29%	0.00%
Hindi	SFT	**42.00%**	**39.86%**
Nepali	Pretrained	0.00%	0.00%
Nepali	SFT	**14.29%**	**10.29%**

Looking at the improvement after fine-tuning we can say that pretraining primarily had language-modeling behaviour, but the synthetic supervised data helped to get a new task-specific response behaviour.

6.2 Performance by Task Family

Table 11: Post-SFT reasoning accuracy by task family.

Task family	Hindi SFT	Nepali SFT
Direct comparison	44%	14%
Largest/smallest	19%	7%
General comparison	39%	18%
Two-hop transitivity	40%	9%
Three-hop transitivity	65%	22%
Reversed relation	46%	17%
Mixed equality	41%	13%

These results say that the task families were not equally easy for the fine-tuned models.

6.3 Error Analysis

Table 12: Error categories on the reasoning test set.

Error category	HI Pre	HI SFT	NE Pre	NE SFT
No recognizable answer	620	248	627	502
Wrong entity	2	158	0	90
Equality relation missing	75	0	73	7
Ambiguous multiple entities	1	0	0	1

The pretrained model mostly failed as it didn't gave a recognizable answer, which is expected since it was only trained on normal text. After SFT, this happened much less often, and also there were some wrong-entity errors.

There were also some cases where the reasoning was correct but the exact answer did not match. This happened in 15 Hindi and 41 Nepali examples.

7 Attention Analysis

All 700 synthetic reasoning test examples were used for this analysis For each model, an early layer (Layer 0) and a late layer (Layer 11) have been used for visualisation. Three heads were selected based on aggregate statistics:

- **Focused:** minimum mean attention entropy;
- **Local:** minimum mean attention distance;
- **Long-range:** maximum mean attention distance.

7.1 Selected Head Identities

Table 13: Representative attention heads before and after SFT.

Language	Layer	Role	Pre	SFT
Hindi	0	Local	H1	H1
Hindi	0	Long-range	H5	H5
Hindi	0	Focused	H4	H2
Hindi	11	Local	H4	H5
Hindi	11	Long-range	H1	H3
Hindi	11	Focused	H0	H0
Nepali	0	Local	H3	H3
Nepali	0	Long-range	H1	H0
Nepali	0	Focused	H2	H2
Nepali	11	Local	H2	H2
Nepali	11	Long-range	H1	H4
Nepali	11	Focused	H3	H3

SFT changed the selected heads in several layers. For example, in Hindi Layer 11, the long-range head changed from H1 in the pretrained model to H3 after SFT. Hindi Layer 11 also changed its local head from H4 to H5. In Nepali, the Layer 0 long-range head changed from H1 to H0, and the late-layer long-range head changed from H1 to H4.

These changes say that fine-tuning altered attention behaviour.

7.2 Attention Heatmaps

The heatmaps use the same test example from the synthetic dataset for both pretrained and SFT comparisons.

The heatmaps here show heads whose local, long-range, or focused roles changed between the pretrained and SFT models. Each comparison shows the corresponding pretrained head beside the SFT head.

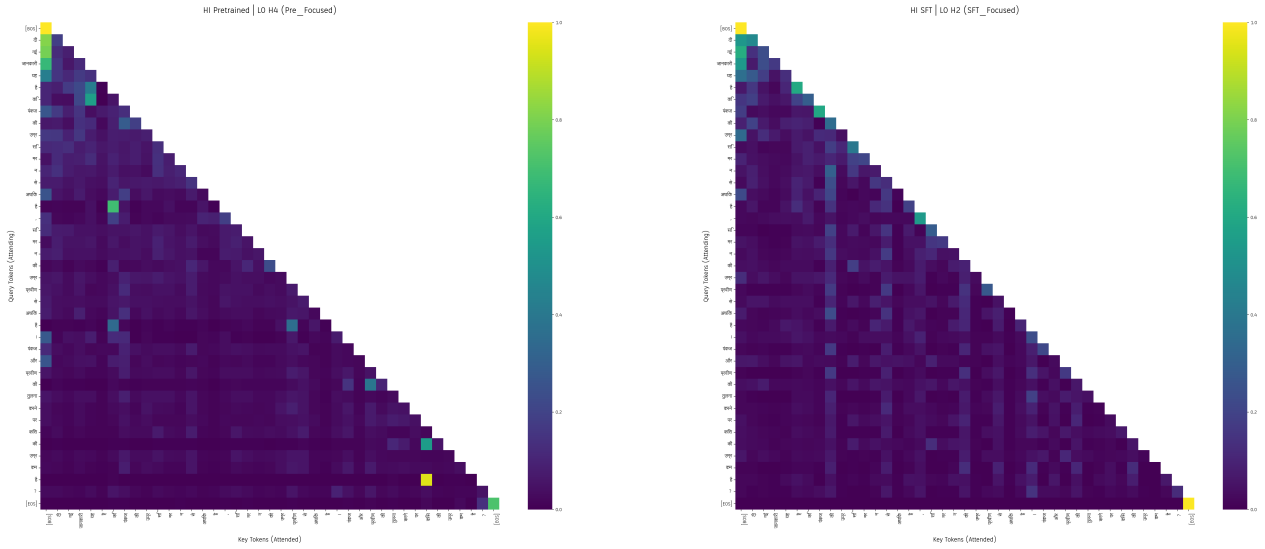
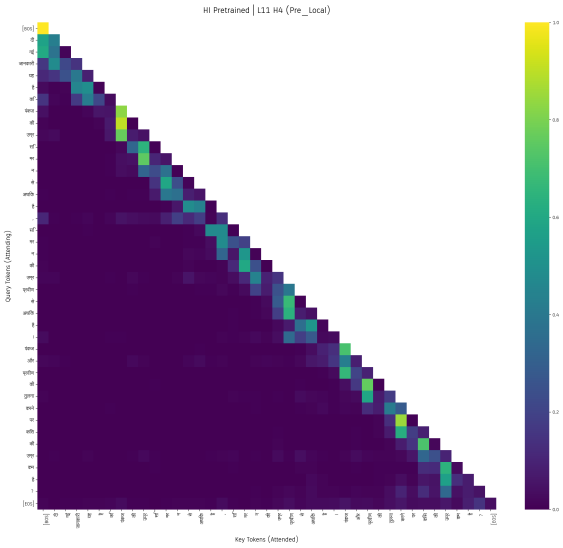
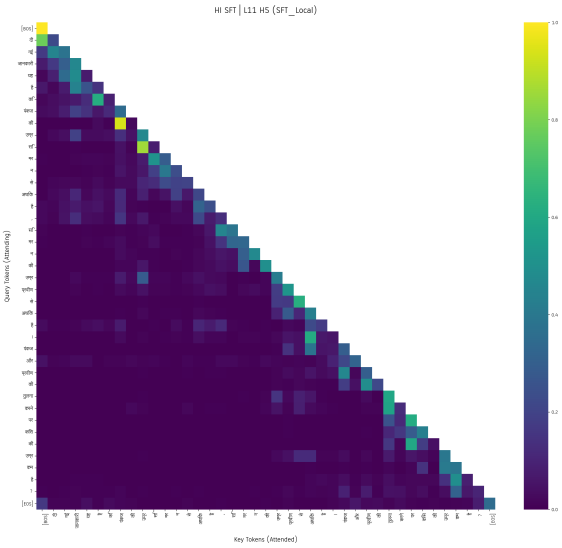


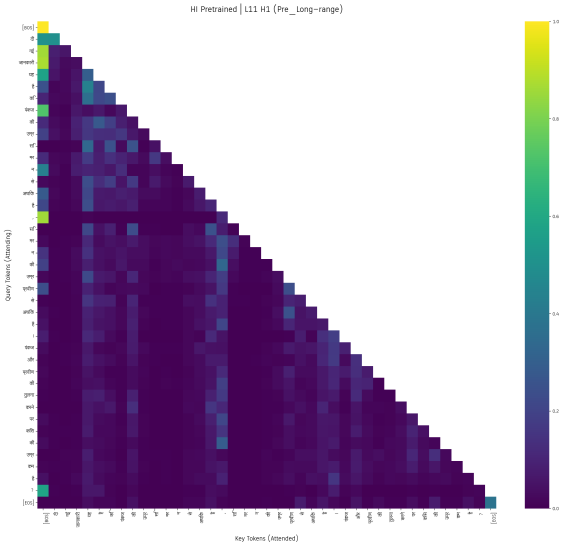
Figure 2: Hindi Layer 0 focused-head comparison before and after SFT. The selected focused head changes from H4 to H2.



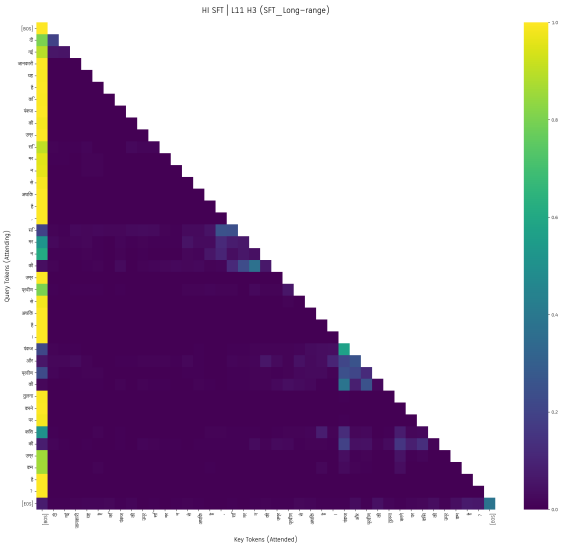
Pretrained – Local H4



SFT – Local H5

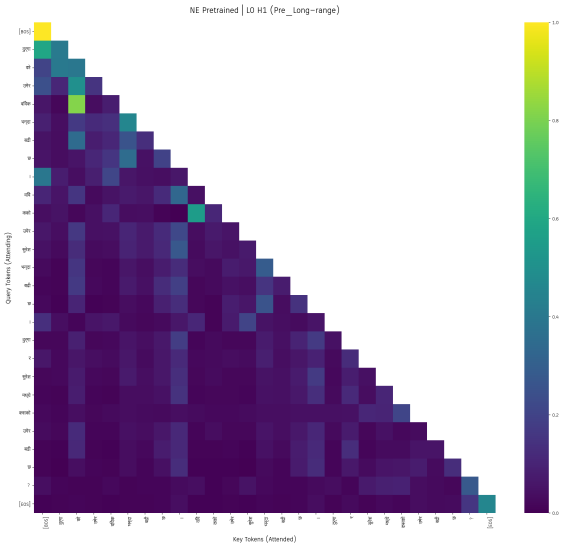


Pretrained – Long-range H1

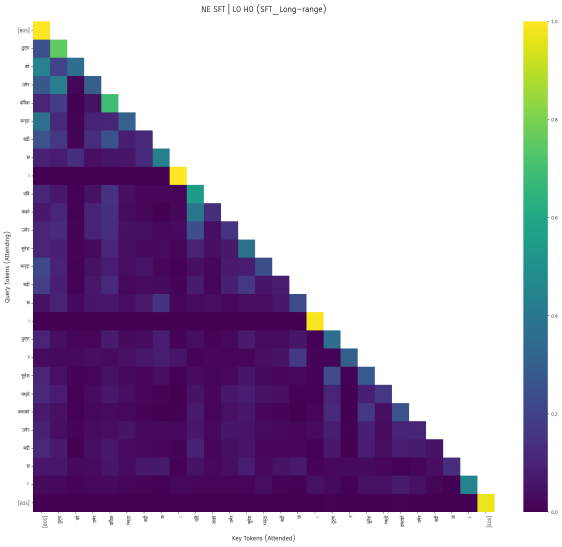


SFT – Long-range H3

Figure 3: Hindi Layer 11 attention-head comparisons before and after SFT. The selected local head changes from H4 to H5, while the selected long-range head changes from H1 to H3.

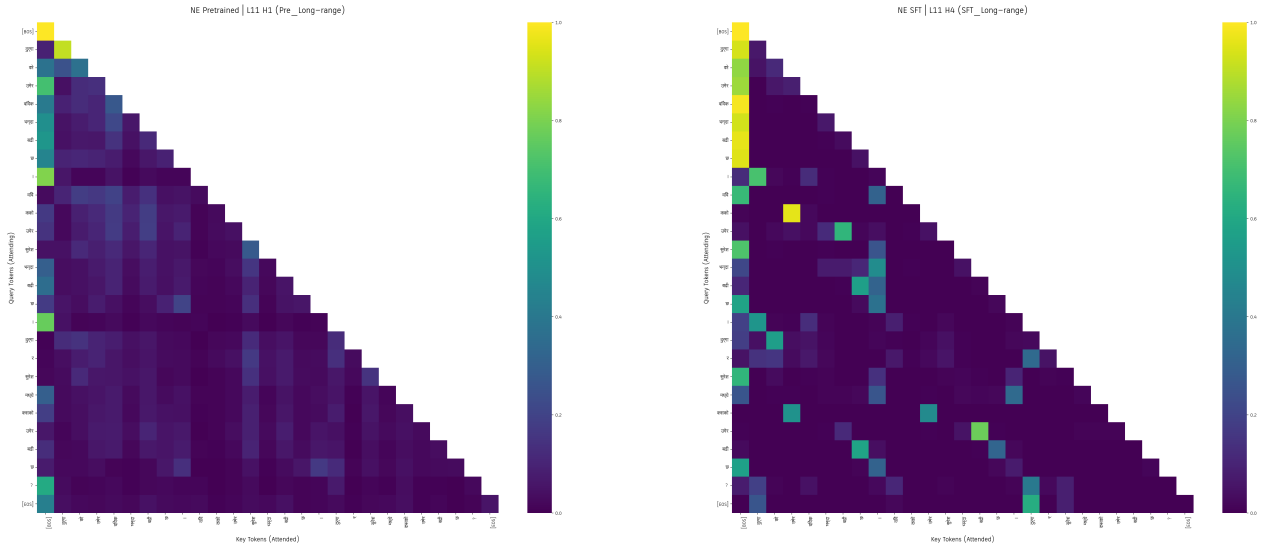


Pretrained – H1



SFT – H0

Figure 4: Nepali Layer 0 long-range-head comparison before and after SFT. The selected long-range head changes from H1 to H0.



Pretrained – H1 SFT – H4

Figure 5: Nepali Layer 11 long-range-head comparison before and after SFT. The selected long-range head changes from H1 to H4.

8 Cross-Language Analysis

8.1 1. How did data scale and quality differ?

Both languages were trained on almost the same amount of data, around 500M tokens each. So, we cant say the difference in performance based on the no of tokens used for training.

The two corpora were different in terms of their data sources. More manual data was available for hindi. For nepali, i had to scrape newspapers to get to the 20% requirement. Hindi had more manually collected data, including Wikipedia and Wikisource, compared to Nepali. After deduplicaton, manual data was 17.74% of Hindi documents and 9.15% of Nepali documents.

The cleaning and deduplication results show that the two datasets were different. Hindi had more cleaning and duplicate removal. These differences were still present even though both languages used about the same number of training tokens.

8.2 2. How do language-modeling and reasoning results compare?

For language modeling, Hindi has a test perplexity of 11.66, while Nepali has 12.51. But Nepali has a lower test BPB of 0.5206 compared with Hindi's 0.6441. So, the two metrics show different results.

For reasoning, Hindi performs much better after SFT, reaching 42.00% accuracy compared with 14.29% for Nepali. Both models improved a lot from almost zero accuracy before SFT, this shows that the SFT data helped the models learn the required reasoning tasks.

So, language modeling and reasoning performance are not the same. Based on a small difference in perplexity we can not directly say or assume a similar difference in reasoning performance.

8.3 3. Which tokenizer and corpus factors affected the lower-resource model?

The main tokenizer difference is fertility. On the test data, Nepali has a fertility of 2.4494, and we have 2.3782 for Hindi. This means Nepali uses more subword tokens for an average word. when we have the same 512-token context, this means that fewer original words can fit into each sequence for Nepali

The bytes per token are also different. Nepali has 7.0014 bytes/token, compared with 5.5019 for Hindi. At the same time, Nepali has a lower BPB (0.5206 vs. 0.6441). This means that Nepali's higher bytes per token does not mean worse language-model performance when we measured per byte. So, comparing only token-level perplexity can be misleading because the two languages use tokens differently. BPB will give a more comparable view when we have two different tokenizations.

Both final datasets used about 500M tokens, with around 20% manual data and 80% public data. But, the original Hindi and Nepali datasets were different in size, sources, and cleaning. These differences may have affected the models. But we can't say surely.

8.4 4. What evidence explains the observed differences?

Several results support this explanation:

1. Both models were trained on almost the same number of tokens and used the same architecture. So, model size and training data amount were the same.
2. Hindi has lower test perplexity, while Nepali has lower BPB. This shows that the tokenizer will affect the language-modeling results.
3. After SFT, Hindi achieves much higher reasoning accuracy than Nepali. Results were not same
4. The error types are also different. Hindi has more wrong-entity errors, while Nepali has more cases where no recognizable answer was produced.
5. Attention patterns also change after SFT in both languages. Some selected heads change their roles, this shows that SFT changes the model's attention behaviour.

Overall, we cant explain the difference by one factor alone. even though both models used the same architecture and almost the same number of training tokens, we have differences in the data and tokenization so that led to different results.